

İSG BİLDİRİM SİSTEMİ

Detaylı Teknik Analiz Raporu

Fabrika İSG — İş Kazası & Ramak Kala Bildirim Sistemi

Proje türü:	Staj / Bitirme projesi — Full-stack web uygulaması
Mimari:	NestJS + Prisma + PostgreSQL React + Vite + TypeScript
Kapsam:	Tüm katmanların teknik incelemesi + 17 iş günü yol haritası
Yapı:	5 Milestone / 8 Sprint (end-to-end)
Belge:	Ana Teknik Analiz Raporu (0/9)
Hazırlık:	Kod tabanının satır düzeyinde incelenmesiyle üretilmiştir

1. Yönetici Özeti

Bu rapor, bir fabrikanın tüm iş kazası ve ramak kala (near-miss) bildirimlerini toplamak, sınıflandırmak, soruşturmak ve kapatmak için geliştirilmiş rol/yetki tabanlı (RBAC) web uygulamasının uçtan uca teknik analizidir. Uygulama iki bağımsız projeden oluşur: bir NestJS REST API (backend) ve bir React tek-sayfa uygulaması (client). Kalıcı veriler PostgreSQL'de, yüklenen foto/videolar ise S3 uyumlu MinIO nesne depolamasında tutulur; tüm servisler Docker ile ayağa kaldırılır.

İncelenen kod tabanı olgun mühendislik alışkanlıkları göstermektedir: granüler izin modeli, global guard zinciri ile korunan uçlar, tek kaynaklı (single source of truth) durum geçiş makinesi, atomik transaction'lar, veri minimizasyonu, imzalı (presigned) ve yetkilendirilmiş dosya erişimi ve kesintisiz denetim (audit) kaydı. Rapor; bu katmanları ayrı ayrı çözümler, projeyi 17 iş gününe yayan gerçekçi bir yol haritası sunar ve her kilometre taşı (milestone) için hem geliştirilecek bileşenleri hem de son kullanıcının göreceği/görmeyeceği noktaları gösteren tasarım şemaları içerir.

Bir Bakışta Proje Metrikleri

Metrik	Değer
Backend kaynak dosyası	~40 TypeScript dosyası (yaklaşık 3.700 satır)
Frontend kaynak dosyası	~32 dosya (yaklaşık 6.500 satır)
Veri modeli	9 ana model + 15+ enum (Prisma)
Granüler izin (Permission)	19 adet — rollere serbestçe atanır
Ana modül (NestJS)	8 (Auth, Users, Roles, Departments, Reports, Notifications, Uploads, Audit)
Bildirim türü	2 (İş Kazası, Ramak Kala) — tek tabloda tip ayrımıyla
Dil desteği	Türkçe / İngilizce (i18next)
Depolama	PostgreSQL 16 (ilişkisel) + MinIO (nesne)

2. Teknoloji Yığını (Technology Stack)

Backend

Katman	Teknoloji	Görevi
Çatı	NestJS 10	Modüler, DI tabanlı sunucu çatısı
Dil	TypeScript 5.6	Statik tipli güvenli geliştirme
ORM	Prisma 5.20	Tip-güvenli veri erişimi + migration
Veritabanı	PostgreSQL 16	İlişkisel kalıcı depolama
Kimlik	JWT + Passport	Token tabanlı oturum
Şifre	bcryptjs	Parola hash'leme
Doğrulama	class-validator	DTO seviyesinde girdi doğrulama
Depolama	MinIO (S3)	Foto/video nesne depolama

Frontend

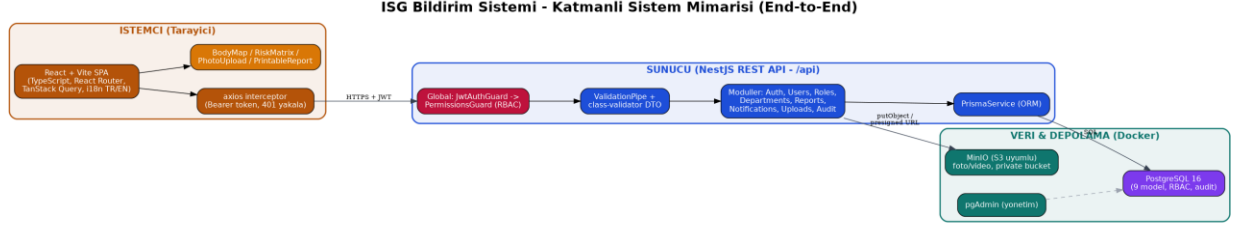
Katman	Teknoloji	Görevi
Çatı	React 18	Bileşen tabanlı UI
Derleyici	Vite 5	Hızlı geliştirme/derleme
Yönlendirme	React Router 6	SPA rota yönetimi
Sunucu durumu	TanStack Query 5	API veri önbellegi/senkron
HTTP	axios	İstek interceptor + token
Çoklu dil	i18next	TR/EN yerelleştirme
PDF	html2pdf.js	Bildirimi yazdırma/PDF

Altyapı

docker-compose.yml üç servisi tek komutla ayağa kaldırır: PostgreSQL (host portu 5433), pgAdmin (web yönetimi, 5050) ve MinIO (S3 API 9000 / konsol 9001). Bucket (hse-uploads) backend ilk açılışta otomatik oluşturulur; sağlık kontrolü (healthcheck) ile servis hazırlığı denetlenir.

3. Sistem Mimarisi (Genel Bakış)

Uygulama klasik üç katmanlı bir mimariyi izler: tarayıcıda çalışan SPA istemci, REST API sunucusu ve veri/depolama katmanı. İstemci ile sunucu arasındaki tüm trafik JWT taşıyıcı (Bearer) token ile korunur; her istek global guard zincirinden geçer.



Şekil 1 — Katmanlı sistem mimarisi (istemci / sunucu / veri)

İstek Yaşam Döngüsü (Request Lifecycle)

1. React bileşeni bir eylem tetikler; axios interceptor isteğe Authorization: Bearer <token> ekler.
2. Vite dev sunucusu /api isteklerini NestJS'e (localhost:3000) yönlendirir (proxy).
3. JwtAuthGuard token'ı doğrular; jwt.strategy kullanıcıyı DB'den taze yükler (rol değişiklikleri anında yansır).
4. PermissionsGuard uçtaki @RequirePermissions ile kullanıcının izinlerini karşılaştırır.
5. Controller, ValidationPipe'tan geçen DTO'yu servise iletir; servis Prisma ile veriye erişir.
6. Kritik işlemler AuditService ile sessizce loglanır; ilgili kullanıcılara in-app bildirim düşer.

4. Katman Katman Teknik Analiz

4.1 Altyapı & DevOps Katmanı

Tek bir docker-compose dosyası tüm bağımlılıkları taşınabilir kılar. PostgreSQL host portu bilinçli olarak 5433'e alınmıştır (yerel 5432 çakışmasını önlemek için). MinIO kök kullanıcı bilgileri ortam değişkeniyle verilir ve bucket private kalır. Bu yaklaşım, geliştiricinin makinesinde 'çalıştır ve unut' türü bir kurulum sağlar.

- docker-compose.yml — PostgreSQL + pgAdmin + MinIO orkestrasyonu, healthcheck ve kalıcı volume'lar.
- pgadmin/servers.json — DB bağlantısını önceden tanımlı getirir; ilk açılış sürtünmesini azaltır.
- .env / .env.example — DATABASE_URL, JWT_SECRET, CORS_ORIGIN, MinIO ve süper admin tohum değerleri.

4.2 Veri Modeli Katmanı (Prisma)

Şema 9 ana modelden oluşur ve iş alanını (domain) titizlikle enum'larla kodlar. En kritik tasarım kararı, iki bildirim türünün (İş Kazası / Ramak Kala) tek bir Report tablosunda 'type' alanıyla ayrılmasıdır (single-table inheritance). Bu, ortak iş akışını (durum, atama, CAPA, ekler) tek yerde tutarken türe özel alanları isteğe bağlı bırakır.

Model	Sorumluluk	Öne çıkan ilişkiler
User	Kullanıcı, durum (PENDING/VERIFIED...), rütbe	Role (N:1), UserDepartment (N:M), Report
Role	İzin listesi taşıyan rol	Permission[] enum dizisi, User
Department	Birim/bölüm	UserDepartment, Report
Report	İş kazası & ramak kala (tek tablo)	reporter, assignedTo, closedBy, department
BodyInjury	Vücut haritası yaralanma noktaları	Report (N:1, cascade)
CorrectiveAction	Düzeltilici/önleyici faaliyet (CAPA)	Report, assignedTo
Notification	Uygulama içi bildirim	user, sender, targetUser, report
AuditLog	Denetim kaydı (kim-ne-ne zaman)	actor (SetNull), actorName anlık saklanır
UserDepartment	Kullanıcı-Birim çokla-çok bağı	User + Department bileşik anahtar

Silme davranışları özenle seçilmiştir: bildirimi olan kullanıcı Restrict ile silinemez (kayıtlar korunur); BodyInjury ve CorrectiveAction Report'a Cascade bağlıdır; AuditLog aktörü SetNull olur ama actorName metni kalıcı saklanır (kullanıcı silinse bile denetim izi bozulmaz).

4.3 Backend Uygulama Katmanı (NestJS)

8 modül tek bir AppModule altında toplanır. Modüller net sorumluluk sınırlarına sahiptir ve bağımlılık enjeksiyonu (DI) ile gevşek bağlıdır. main.ts global '/api' öneki, CORS ve whitelist'li ValidationPipe kurar — böylece DTO'da tanımlı olmayan alanlar sessizce ayıklanır.

Modül	Ne yapar
Auth	Kayıt, giriş, JWT üretimi, /me, profil ve şifre değişikliği
Users	Onay iş akışı, rol/rütbe/birim atama, askıya alma, atanabilir kullanıcılar
Roles	Rol ve granüler izin (Permission) yönetimi; sistem rolleri korunur
Departments	Birim CRUD (3 karakter kod kuralı ile)
Reports	Bildirim oluşturma, listeleme (kapsamlı), durum, atama, CAPA, istatistik
Notifications	İzin bazlı toplu bildirim, okundu/okunmadı yönetimi
Uploads	Foto/video yükleme (MinIO'ya) ve güvenli sunum (files.controller)
Audit	Kesintisiz denetim kaydı (hata olsa bile ana akışı bozmaz)

4.4 Güvenlik & Yetkilendirme Katmanı (RBAC)

Güvenlik modeli üç kavram üzerine kuruludur: 19 granüler Permission enum'u, bunları taşıyan Role ve bilgilendirme/sıralama amaçlı Rank (rütbe). İki global guard tüm uçları korur: önce JwtAuthGuard (kimlik), sonra PermissionsGuard (yetki).

- **jwt.strategy** her istekte kullanıcıyı DB'den okur — rol/izin değişikliği anında etkilidir, token'ı beklemez. Taze yetki:
- **tüm izinlere sahiptir ve reddedilemez/askıya alınamaz (guard içinde kısa devre).** Süper admin:
- **status != VERIFIED** olan kullanıcı yetki kullanamaz; anlaşılır Türkçe hata döner. Onay kapısı:
- **listelemede kullanıcı yalnızca kendi/biriminin/atandığı bildirimleri görür (REPORT_VIEW_ALL hariç).** Kapsam filtresi (scope):
- **report-access.ts** tek yerde tanımlıdır; hem detay ucu hem güvenli dosya sunumu aynı kuralı kullanır. Görünürlük kuralı:

4.5 Frontend Katmanı (React + Vite)

İstemci, kimlik durumuna göre üç farklı deneyim sunar: giriş yapılmamış (login/register), onay bekleyen (pending) ve tam uygulama. AuthContext /me çağrısıyla kullanıcı profilini ve izinlerini tutar; Layout, menü öğelerini kullanıcının izinlerine göre koşullu gösterir (hasPermission yardımcısı).

- **Login, Register, Pending, Dashboard, Reports (liste/detay), Accident/NearMiss formları, Admin (Users/Roles/Departments/Audit), Profile.** Sayfalar:
- **BodyMap (tıklanabilir SVG vücut haritası), RiskMatrix (5×5 canlı önizleme), PhotoUpload, AuthenticatedMedia (imzalı medya), PrintableReport (PDF), NotificationBell.** İmza bileşenleri:
- **axios interceptor 401'de oturumu temizler; TanStack Query önbellek ve yeniden-deneme yönetir.** Dayanıklılık:

4.6 Nesne Depolama & Güvenli Dosya Katmanı

Yüklenen medya MinIO'da private bir bucket'ta tutulur. Kritik güvenlik kararı: dosyalar herkese açık değildir. files.controller her erişimde (a) anahtar biçimini doğrular, (b) anahtarın gerçekten bir bildirimin ekinde geçtiğini kontrol eder ve (c) kullanıcının o bildirimi görme yetkisini report-access kuralıyla sınar. Video için HTTP Range (206 Partial Content) desteği vardır; ayrıca en fazla 5 dakika geçerli imzalı URL üretilebilir ve bu URL hiçbir yere loglanmaz/yazılmaz.

4.7 Gözlemlenebilirlik & Denetim Katmanı

AuditService her kritik işlemde (giriş, onay, rol değişikliği, bildirim durum değişikliği, kapatma, dosya vb.) kim-ne-ne zaman kaydı yazar. Tasarım gereği audit yazımı başarısız olsa bile ana iş akışı bozulmaz (try/catch ile yutulur). Dashboard, kapsam-farkında groupBy sorgularıyla türe, duruma ve risk seviyesine göre özet üretir.

5. İmza Özelliklerin Derin İncelemesi

5x5 Risk Matrisi (Ramak Kala)

riskScore = severity × likelihood formülüyle 1–25 arası bir skor üretilir ve eşiklerle seviyeye çevrilir: LOW (≤4), MEDIUM (≤9), HIGH (≤15), CRITICAL (>15). Aynı mantık hem istemcide canlı önizleme için hem de backend'de (computeRisk) yeniden hesaplanır — istemciye güvenilmez, otorite backend'dir.

```
static computeRisk(severity, likelihood) {  
  const score = severity * likelihood;  
  let level; if (score≤4) level='LOW'; else if(score≤9) level='MEDIUM';  
  else if (score≤15) level='HIGH'; else level='CRITICAL';  
  return { score, level };  
}
```

Tıklanabilir Vücut Haritası (İş Kazası)

Ön/arka SVG insan figürü üzerinde bölgeye tıklanır; her yaralanma bodyPart + side + view + type + severity beşlisi olarak kaydedilir. Bu, serbest metne göre yapılandırılmış, raporlanabilir ve istatistiğe uygun veri üretir.

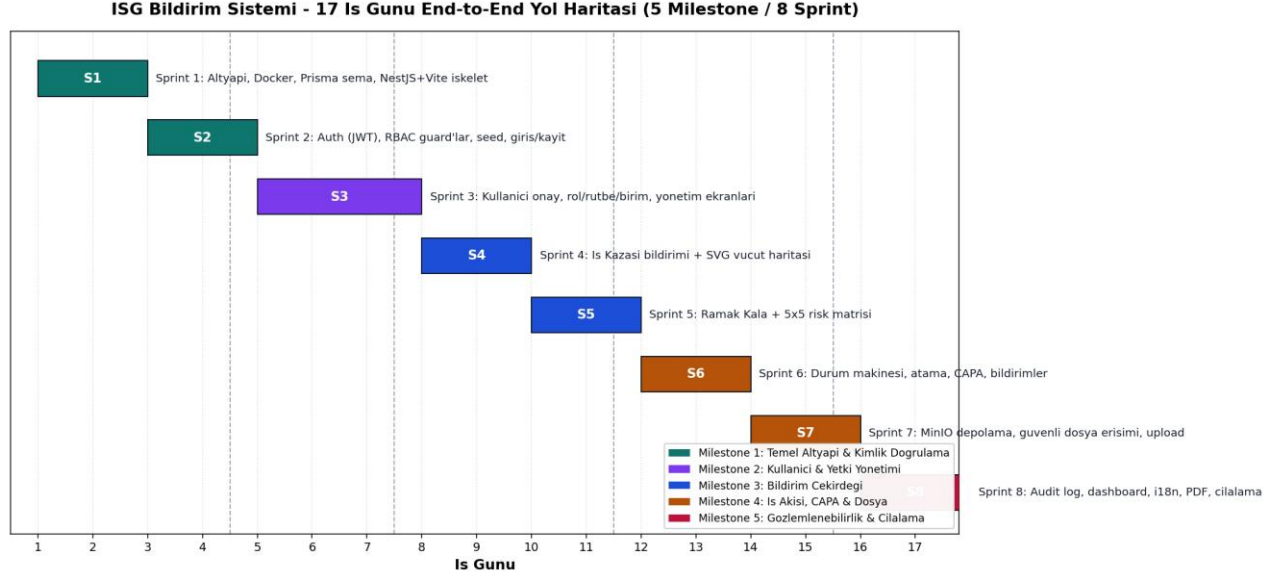
Durum Geçiş Makinesi (Tek Kaynak)

report-status.ts, izin verilen tüm geçişleri tek bir tabloda tutar. CLOSED ve REJECTED son durumlardır (yeniden açılmaz). Kapatma için: en az 10 karakter açıklama zorunludur ve bağlı tüm düzeltici faaliyetler VERIFIED olmalıdır. Okuma-doğrulama-yazma tek transaction içinde yapılır; böylece kontrol ile güncelleme arasında durum kayamaz.

Durum	Geçebileceği durumlar
DRAFT	SUBMITTED
SUBMITTED	UNDER_REVIEW, INVESTIGATING, REJECTED
UNDER_REVIEW	INVESTIGATING, REJECTED
INVESTIGATING	ACTIONS_PENDING, CLOSED, REJECTED
ACTIONS_PENDING	INVESTIGATING, CLOSED
CLOSED / REJECTED	(son durum — geçiş yok)

6. 17 İş Günü End-to-End Yol Haritası

Aşağıdaki plan, mevcut kod tabanının bir staj süresince (17 iş günü ≈ 3,5 hafta) sıfırdan nasıl inşa edileceğini gerçekçi bir sıralamayla gösterir. Bağımlılıklar gözetilerek önce temel (altyapı+kimlik), sonra yönetim, ardından çekirdek bildirim, iş akışı/dosya ve son olarak gözlemlenebilirlik/cılalama gelir.



Şekil 2 — 17 iş günü Gantt: 5 milestone / 8 sprint dağılımı

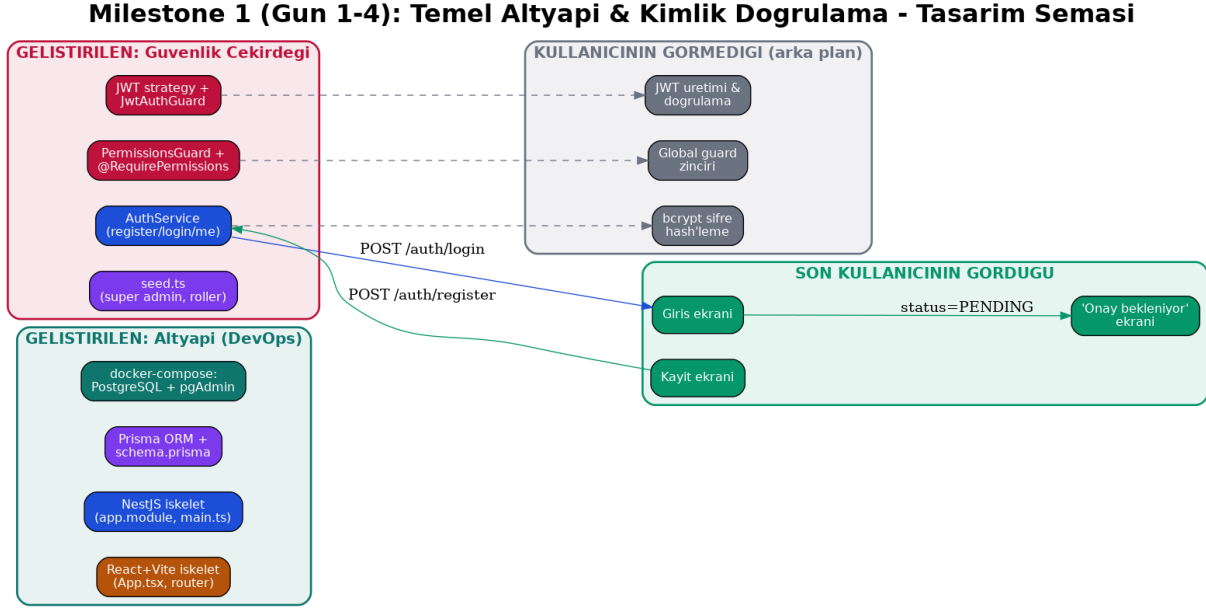
Milestone	Gün	Sprint(ler)	Odak
M1 — Temel Altyapı & Kimlik	1–4	S1, S2	Docker, şema, NestJS/Vite iskelet, Auth+RBAC
M2 — Kullanıcı & Yetki Yönetimi	5–7	S3	Onay iş akışı, rol/rütbe/birim, yönetim ekranları
M3 — Bildirim Çekirdeği	8–11	S4, S5	İş kazası + vücut haritası, ramak kala + 5x5 risk
M4 — İş Akışı, CAPA & Dosya	12–15	S6, S7	Durum makinesi, atama, CAPA, MinIO güvenli dosya
M5 — Gözlemlenebilirlik & Cılalama	16–17	S8	Audit, dashboard, i18n, PDF, mobil, son rötuşlar

7. Milestone Tasarım Şemaları (Teknik + Kavramsal)

Her şema üç perspektifi bir arada gösterir: (1) o milestone'da GELİŞTİRİLEN bileşenler, (2) SON KULLANICININ GÖRDÜĞÜ arayüz noktaları, (3) KULLANICININ GÖRMEDİĞİ arka plan mantığı ve veri modeli. Kesik oklar 'arka planda gerçekleşir' anlamındadır.

Milestone 1 — Temel Altyapı & Kimlik Doğrulama (Gün 1–4)

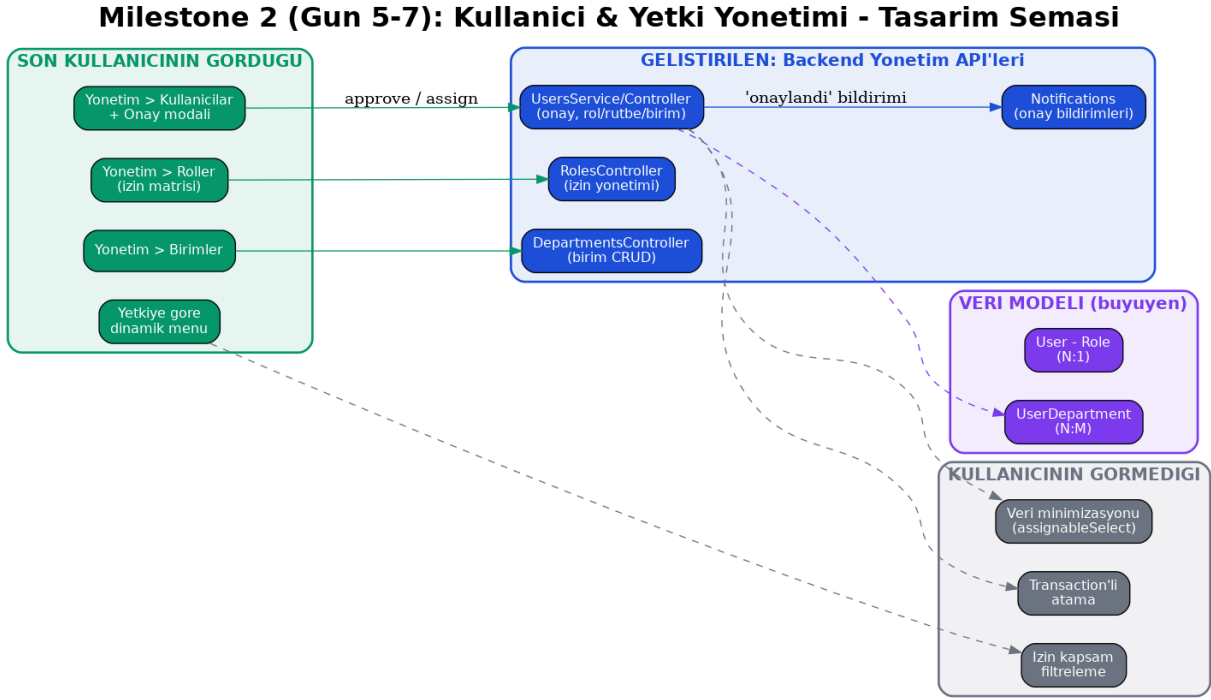
Projenin iskeleti ve güvenlik çekirdeği kurulur. Kullanıcı yalnızca giriş/kayıt/onay-bekleme ekranlarını görür; bcrypt hash'leme, JWT üretimi ve global guard zinciri tamamen arka plandadır.



Şekil — Milestone 1 — Temel Altyapı & Kimlik Doğrulama (Gün 1–4) tasarım şeması

Milestone 2 — Kullanıcı & Yetki Yönetimi (Gün 5–7)

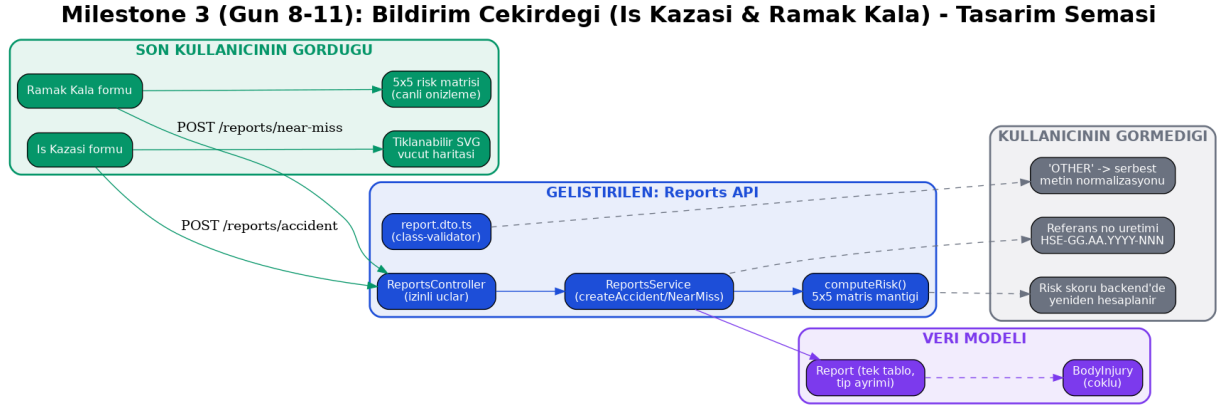
Yönetim API'leri ve ekranları eklenir. Kullanıcı, yetkisine göre değişen bir yönetim menüsü ve onay modali görür; kapsam filtreleme, veri minimizasyonu ve transaction'lı atama görünmez kalır.



Şekil — Milestone 2 — Kullanıcı & Yetki Yönetimi (Gün 5–7) tasarım şeması

Milestone 3 — Bildirim Çekirdeği (Gün 8–11)

Asıl iş değeri burada üretilir: iş kazası (vücut haritası) ve ramak kala (5x5 risk) formları. Referans no üretimi, riskin backend'de yeniden hesaplanması ve 'OTHER' normalizasyonu gizlidir.

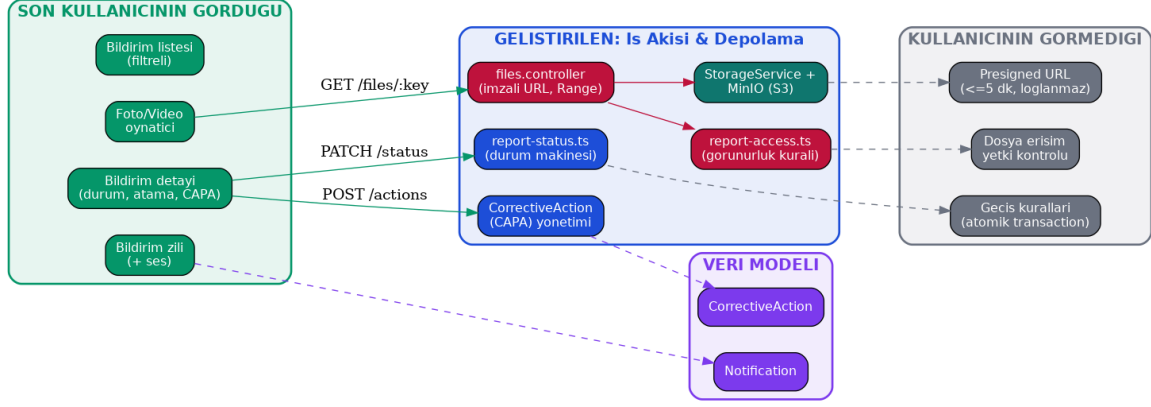


Şekil — Milestone 3 — Bildirim Çekirdeği (Gün 8–11) tasarım şeması

Milestone 4 — İş Akışı, CAPA & Güvenli Dosya (Gün 12–15)

Bildirimin yaşam döngüsü (durum, atama, CAPA) ve güvenli medya yönetimi eklenir. Atomik geçişler, imzalı URL ve dosya erişim yetki kontrolü kullanıcıya görünmez.

Milestone 4 (Gun 12-15): Is Akisi, CAPA & Guvenli Dosya Yonetimi - Tasarim Semasi

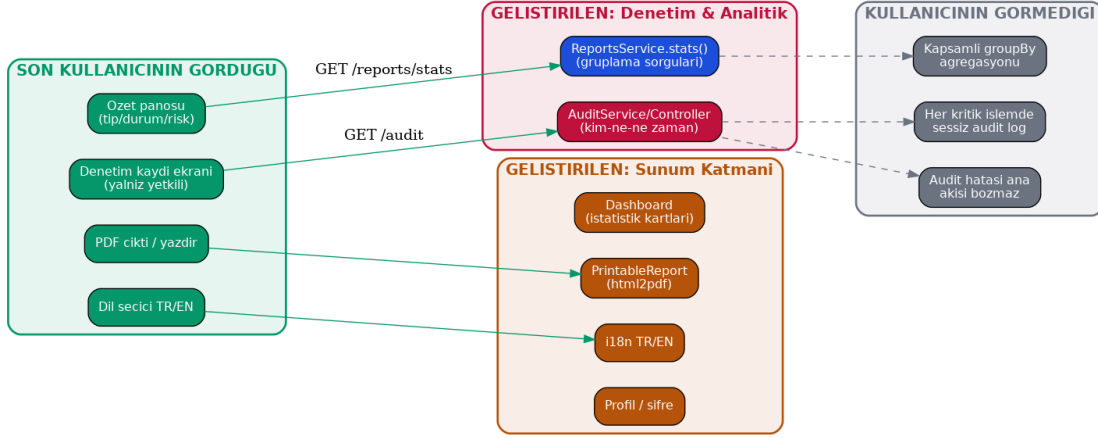


Şekil — Milestone 4 — İş Akışı, CAPA & Güvenli Dosya (Gün 12–15) tasarım şeması

Milestone 5 — Gözlemlenebilirlik, Raporlama & Cilalama (Gün 16–17)

Denetim, pano, PDF çıktı ve çoklu dil ile ürün tamamlanır. Her işlemdeki sessiz audit log ve kapsamlı groupBy agregasyonu perde arkasındadır.

Milestone 5 (Gun 16-17): Gozlemlenebilirlik, Raporlama & Cilalama - Tasarim Semasi



Şekil — Milestone 5 — Gözlemlenebilirlik, Raporlama & Cilalama (Gün 16–17) tasarım şeması

8. Güçlü Yönler, Riskler ve Öneriler

Güçlü Yönler

- Tek kaynaklı durum makinesi ve görünürlük kuralı — kopya mantık yok, bakımı kolay.
- Atomik transaction'lar ile yarış koşullarına (race condition) karşı koruma.
- Private bucket + yetkilendirilmiş, imzalı dosya erişimi — güçlü bir güvenlik duruşu.
- Granüler RBAC ve taze izin okuma — esnek ve güvenli yetkilendirme.
- Kesintisiz, kullanıcı silinse bile bozulmayan denetim kaydı.

Riskler / Geliştirme Alanları

- Otomatik test kapsamı görünmüyor — birim/e2e testleri eklenmeli (öneri: Jest + Supertest).
- JWT_SECRET üretimde mutlaka güçlü ve gizli olmalı; token süre/iptal politikası netleştirilmeli.
- Bildirim listelemede sayfalama (pagination) yok — veri büyüdükçe eklenmeli.
- CorrectiveAction 'OVERDUE' durumu için zamanlanmış iş (cron) tanımlanmalı.
- Sağlık/hazırlık (health) uçları ve yapılandırılmış loglama (ör. pino) izlemeyi kolaylaştırır.

Öneriler

Staj değerlendirmesi açısından proje, bir junior geliştiriciden beklenenin üzerinde bir mimari olgunluk sergiliyor. Bir sonraki adım olarak test altyapısı, CI hattı (GitHub Actions) ve temel gözlemlenebilirlik (health + metrics) eklemek, projeyi 'üretime hazır' seviyesine taşır.

9. Sonuç

İSG Bildirim Sistemi, iş sağlığı ve güvenliği süreçlerini dijitalleştiren, güvenlik ve veri bütünlüğü kararları özenle verilmiş, tam işlevsel bir full-stack uygulamadır. Bu rapor boyunca sunulan 17 günlük yol haritası, 5 milestone ve 8 sprint; hem projenin nasıl inşa edildiğini anlatır hem de her sprint için ayrı ayrı üretilen ayrıntılı sprint raporlarına ve büyüyen entity-relation haritalarına bir çatı oluşturur.